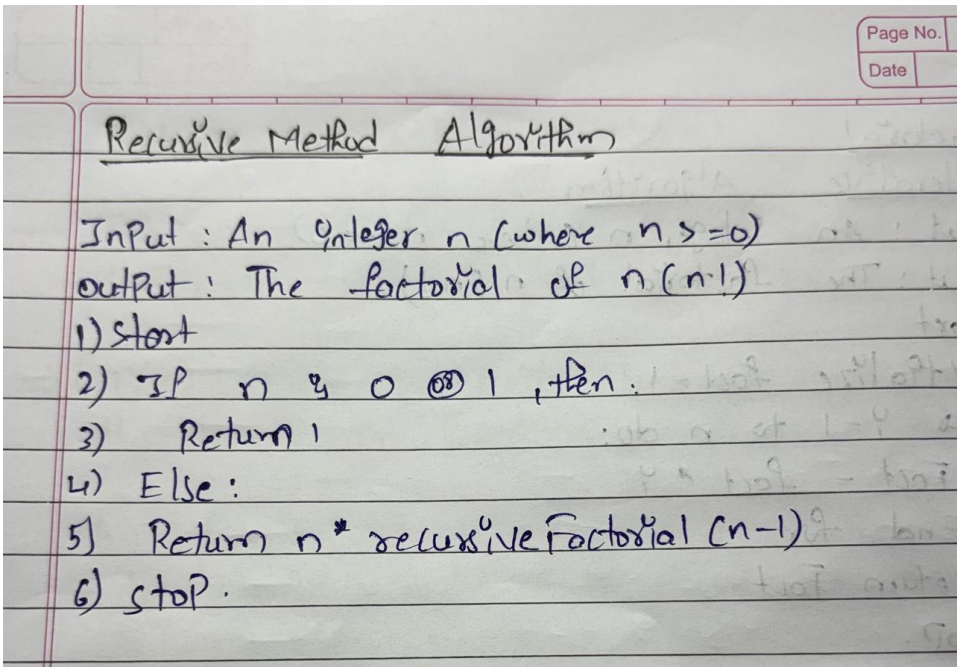
Practical - 4

Aim : To implement and analyze the execution time of the factorial algorithm using both iterative and recursive approaches

1) Recursive Method :



Program / Code :

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int factr(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }
```

```
else{
        int result = n * factr(n - 1);
        return result;
    }
  }
};
int main()
{
  int n, result;
  // Create an object of the class
  Factorial obj;
  cout << "Enter the value to find its factorial: ";
  cin >> n;
  result = obj.factr(n);
  cout << "Factorial of " << n << " is " << result << endl;
  return 0;
}
```

Output :

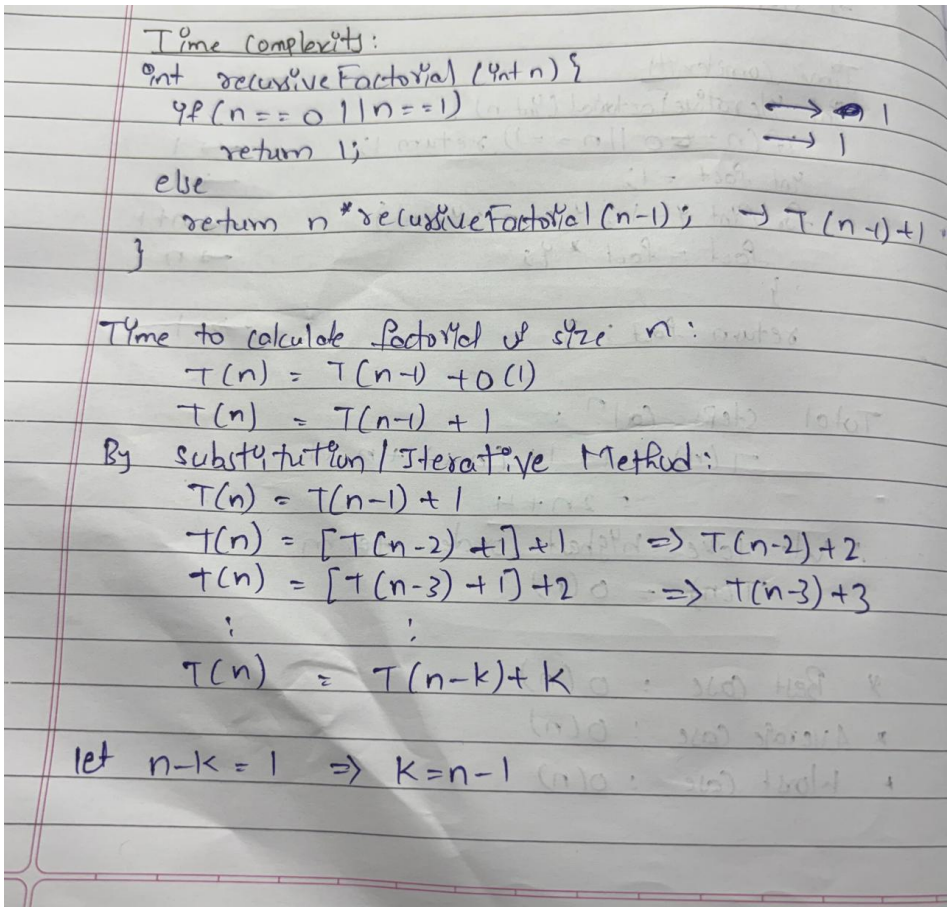
```
C:\Users\NIKSHITH>cd /d C:\Users\NIKSHITH\DAA

C:\Users\NIKSHITH\DAA>g++ -o factorial "Fact Recursive method.cpp"

C:\Users\NIKSHITH\DAA>factorial
Enter the value to find its factorial: 10
Factorial of 10 is 3628800

C:\Users\NIKSHITH\DAA>
```

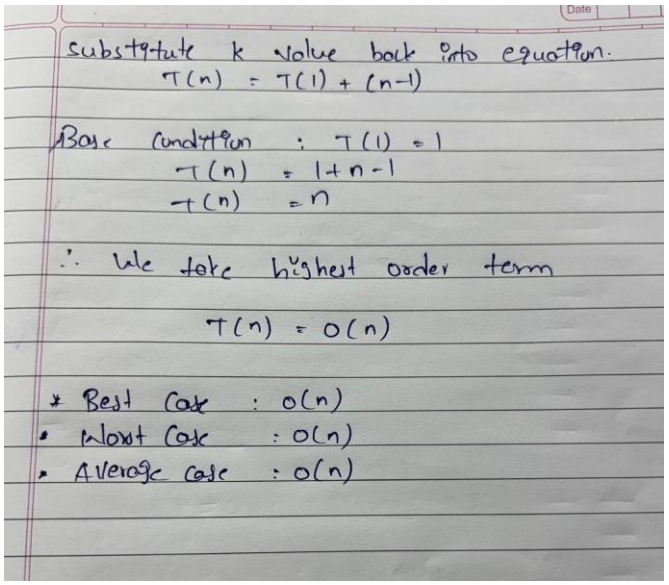
Time Complexity :

Best Case : O(n)

Average Case : O(n)

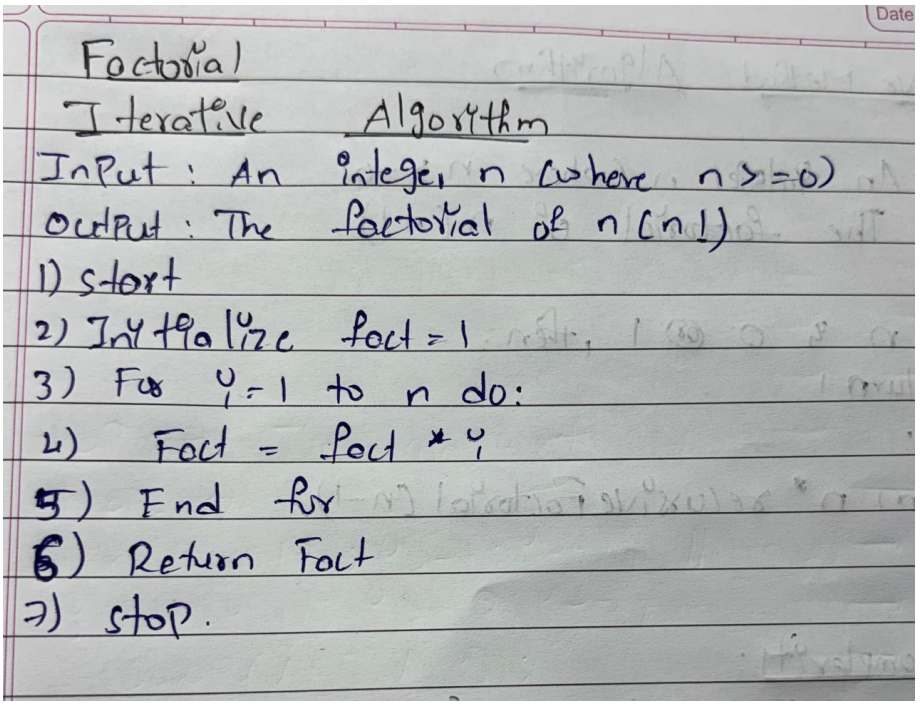
Worst Case : O(n)





2) Iterative Method :

Algorithm :



Program / Code :

```
#include <iostream>
using namespace std;
class Factorial
{
public:
    int fact(int n)
    {
        int result = 1;
        for (int i = 1; i <= n; i++)
        {
            result = result * i;
        }
```

```
 return result;
    }
};
int main()
{
    int n,result;
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
    result = obj.fact(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output :

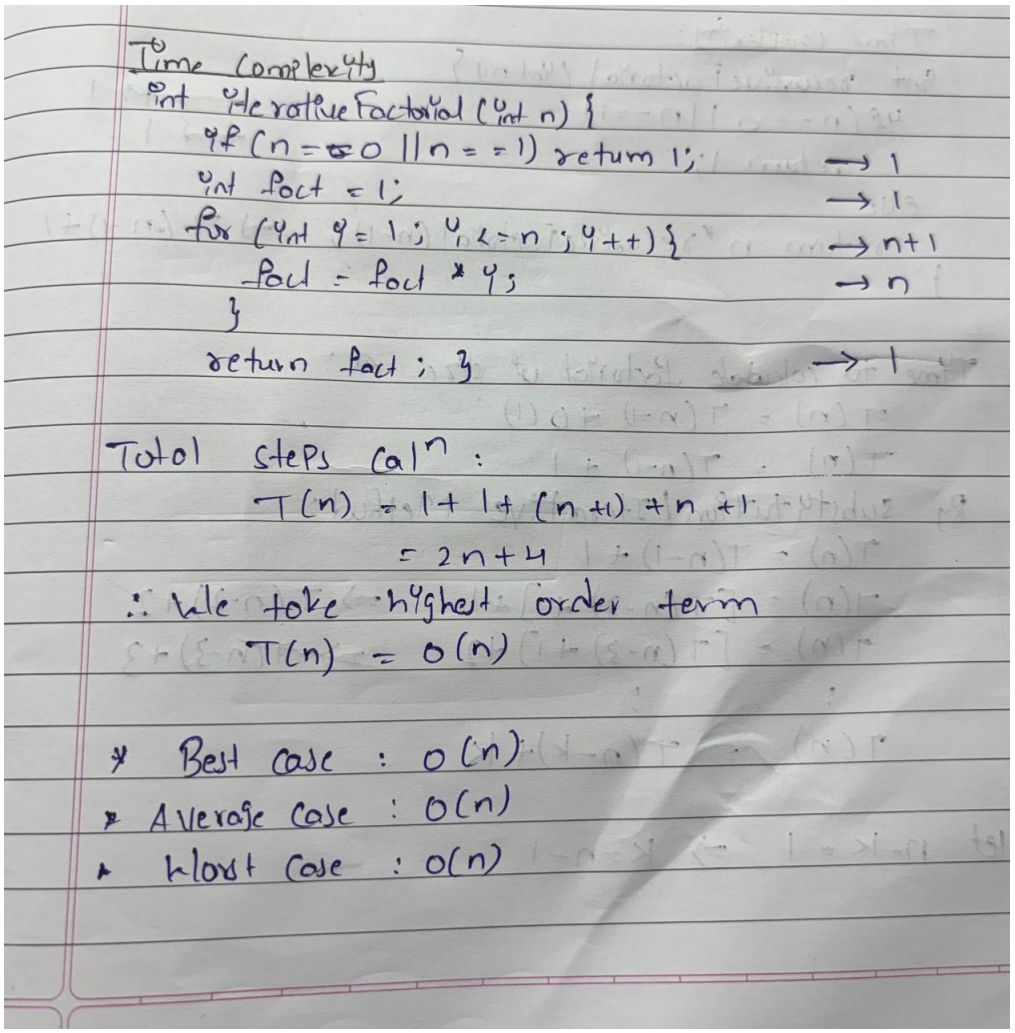
```
C:\Users\NIKSHITH\DAA>
C:\Users\NIKSHITH\DAA>cd /d C:\Users\NIKSHITH\DAA

C:\Users\NIKSHITH\DAA>g++ -o iterative "Fact Iterative method.cpp"

C:\Users\NIKSHITH\DAA>iterative
Enter the value to find its factorial: 7
Factorial of 7 is 5040
```

Time complexity :

Best Case : O(n)

Average Case : O(n)

Worst Case : O(n)



Time Complexity

```
int IterativeFactorial(int n) {
    if (n == 0 || n == 1) return 1;        → 1
    int fact = 1;                          → 1
    for (int i = 1; i <= n; i++) {         → n+1
        fact = fact * i;                   → n
    }
    return fact; }                         → 1
```

Total steps cal^n :

$T(n) = 1 + 1 + (n+1) + n + 1$

$= 2n + 4$

∴ We take highest order term

$T(n) = O(n)$

* Best case : $O(n)$
* Average case : $O(n)$
* Worst case : $O(n)$

Conclusion : The iterative method is faster and more reliable because it uses a direct loop. The recursive method is slower because managing multiple function calls takes more time.